

Transformer-Driven Numerical Logic via SLMs

Rupen Parthu, Ankit Swami, Nikhil Sai

Supervisor: Dr. K Swanthana

Department of Computer Science and Information Technology

KL University, Hyderabad (KLH)

Abstract

Mathematical reasoning is a fundamental capability required in intelligent tutoring systems and scientific computing. While large language models (LLMs) demonstrate strong proficiency in mathematical problem-solving, their high computational requirements inhibit deployment in resource-constrained educational environments. This paper presents MathSLM, a lightweight, decoder-only Transformer model designed specifically for step-by-step mathematical reasoning. Trained via next-token prediction on datasets such as GSM8K and the Hendrycks MATH corpus, MathSLM generates intermediate reasoning steps prior to final answer formulation. We outline the data preprocessing pipeline, tokenization strategies using Byte Pair Encoding (BPE), and architectural configurations. Furthermore, we explore deployment feasibility through a case study on intelligent concurrency patterns for edge AI devices, highlighting the model’s low-latency inference capabilities. Evaluation metrics including perplexity and token prediction accuracy demonstrate that domain-focused Small Language Models can achieve competitive mathematical reasoning capabilities with significantly reduced parameter counts.

Keywords: Small Language Models, Transformer, Mathematical Reasoning, PyTorch, Edge AI.

1 INTRODUCTION

1.1 Background

The advent of autoregressive Transformer architectures has revolutionized natural language processing. In recent years, the application of these models to formal logic and mathematical reasoning has shown immense promise. However, state-of-the-art systems often rely on massive parameter counts, requiring cluster-grade hardware for both training and inference.

1.2 Problem Statement

Despite the success of LLMs, their computational demands and operational costs make them unsuitable for localized, resource-constrained applications, such as offline educational platforms or embedded tutoring devices. There is a critical need for smaller, highly optimized models that retain mathematical reasoning capabilities without the overhead of generalized large models.

1.3 Research Gap

While general-purpose Small Language Models (SLMs) exist, they often struggle with the rigid logical syntax and multi-step derivations required for mathematics. Current research lacks a focused investigation into how aggressively a decoder-only architecture can be scaled down when exclusively trained on a high-quality, domain-specific mathematical corpus.

1.4 Research Objectives

1. Develop a lightweight, decoder-only Transformer tailored for mathematical reasoning.
2. Implement a robust preprocessing and tokenization pipeline for mathematical syntax.
3. Optimize inference generation strategies (greedy, temperature, top-k).
4. Evaluate performance using perplexity and token prediction accuracy.

1.5 Research Questions

- How does domain-specific training on datasets like GSM8K affect the reasoning capabilities of SLMs compared to general-purpose training?
- What is the optimal balance between model depth and attention heads for mathematical token prediction?

1.6 Contributions

This paper contributes a complete architectural framework for MathSLM, an implementation pipeline utilizing PyTorch, and a novel case study on deploying this architecture efficiently using intelligent concurrency patterns for edge AI devices.

1.7 Organization of the Paper

Section 2 reviews related work. Section 3 details the proposed methodology and architectural design. Section 4 outlines the experimental setup. Section 5 presents the quantitative and qualitative results. Section 6 discusses the findings, followed by limitations in Section 7 and conclusions in Section 8.

2 RELATED WORK

2.1 Existing Approaches

Historically, mathematical problem solving in AI relied on symbolic computation and rule-based expert systems. These approaches were highly accurate but lacked the flexibility to understand natural language problem descriptions.

2.2 Machine Learning Approaches

Early machine learning models utilized recurrent neural networks (RNNs) for sequence-to-sequence translation of math word problems to equations, but struggled with long-range dependencies in complex reasoning chains.

2.3 Deep Learning Approaches

The shift toward Transformer-based models introduced the capability to maintain context over longer sequences. Models such as GPT-4 and Claude excel at this, utilizing Chain-of-Thought (CoT) prompting. Simultaneously, SLM architectures like Qwen have demonstrated that high parameter efficiency is achievable.

2.4 Explainability / Existing Techniques

Explainability in mathematical LLMs is inherently tied to the intermediate reasoning steps generated. By forcing the model to output step-by-step logic before the final answer, the reasoning process becomes transparent and auditable.

2.5 Critical Comparison

Unlike general-purpose models (e.g., LLaMA, Qwen) which allocate parameter capacity to disparate knowledge domains, MathSLM restricts its vocabulary and attention mechanisms purely to mathematical constructs, offering higher density learning for this specific domain.

2.6 Research Gap

There remains a gap in establishing standardized, efficient execution frameworks for mathematical SLMs on consumer hardware, particularly concerning concurrent request handling.

3 PROPOSED METHODOLOGY

3.1 Overall Framework

MathSLM employs a standard decoder-only Transformer architecture. The framework processes natural language math problems, generating sequential logical steps that culminate in a final numerical answer.

3.2 Dataset

The model is trained on publicly available datasets including GSM8K (Grade School Math) and Hendrycks MATH. These datasets provide a combination of problem statements, step-by-step reasoning paths, and final solutions.

3.3 Data Preprocessing

Raw data is formatted into a standardized prompt structure combining the question and the solution sequence. Text normalization is applied to standardize mathematical symbols.

3.4 Feature Engineering

In the context of autoregressive language modeling, feature engineering is handled via Byte Pair Encoding (BPE), mapping frequent mathematical sub-strings into distinct token representations.

3.5 Feature Selection

Feature selection is analogous to the vocabulary pruning process, where the tokenizer vocabulary is optimized to represent mathematical operators efficiently without excessive fragmentation.

3.6 Proposed Model

MathSLM consists of an embedding layer, positional encoding, multiple stacked Transformer decoder blocks (each containing multi-head self-attention and a feed-forward network), and a final language model head for next-token prediction.

3.7 Baseline Models

MathSLM is compared against base variants of existing SLMs (e.g., GPT-2 Small, early-stage Qwen iterations) operating under zero-shot constraints.

3.8 Explainability Method

The model’s interpretability is derived from its forced autoregressive generation of intermediate logic, allowing manual verification of each computational step.

3.9 Algorithm / Mathematical Formulation

The core mechanism relies on scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

where Q, K, V represent the query, key, and value matrices, and d_k is the dimensionality of the keys. The model optimizes the cross-entropy loss over the predicted token sequence.

4 EXPERIMENTAL SETUP

4.1 Hardware and Software

The implementation utilizes the PyTorch deep learning framework. Tokenization is managed via the Hugging Face library.

4.2 Dataset Split

Datasets are partitioned into 80% training, 10% validation, and 10% testing splits to ensure rigorous evaluation of generalization capabilities.

4.3 Hyperparameter Configuration

Configurable generation strategies include greedy decoding, temperature sampling (scaling logits prior to softmax), and top-k sampling to dictate token selection determinism.

4.4 Cross-Validation

Iterative validation is performed across dataset subsets to monitor for overfitting during the training epochs.

4.5 Evaluation Metrics

Primary metrics include Training/Validation Loss, Perplexity, and Token Prediction Accuracy. Furthermore, correctness of the final mathematical answer is evaluated automatically against the dataset ground truth.

4.6 Statistical Testing

Variance in generation accuracy across different temperature settings is analyzed to determine the optimal inference configuration.

5 RESULTS

5.1 Overall Performance

Initial evaluations indicate that domain-focused training significantly accelerates convergence on mathematical syntax compared to generalized pre-training.

5.2 Model Comparison

MathSLM demonstrates lower perplexity on the GSM8K test split relative to baseline generalized models of equivalent parameter size.

5.3 Confusion Matrix

(Applicable for token-level classification tasks; in this generative context, analysis focuses on common reasoning divergence points).

5.4 ROC/PR Analysis

Given the generative nature of the task, standard ROC analysis is substituted with exact-match accuracy scaling across varying reasoning depths.

5.5 Feature Importance

Attention map visualizations confirm that the model correctly attends to numerical operators and preceding quantitative values during calculation steps.

5.6 Explainability Results

Qualitative review of generated outputs shows coherent step-by-step logic, though complex arithmetic operations occasionally result in hallucinated values despite correct logical formatting.

5.7 Ablation Study

Reducing the number of attention heads results in a sharp decline in the model’s ability to track long-term dependencies in multi-step word problems.

5.8 Statistical Results

The model achieves baseline competency, proving the viability of the SLM approach for specific mathematical domains.

6 DISCUSSION

6.1 Interpretation of Findings

The results validate the hypothesis that a smaller, specialized Transformer can internalize mathematical reasoning patterns if the training data is highly curated and formatted for Chain-of-Thought generation.

6.2 Comparison With Previous Studies

Consistent with recent literature, we find that generating intermediate steps acts as a computational buffer, significantly improving final answer accuracy over direct-answer prediction.

6.3 Theoretical Implications

The architecture proves that logical reasoning is not strictly an emergent property of massive scale, but can be induced in smaller models through targeted next-token prediction on structured data.

6.4 Practical Implications

Deploying these models requires robust system architecture. As part of this research, a case study on **Intelligent Concurrency Patterns for Edge AI Devices** was conducted. By applying optimized concurrency models, MathSLM can process multiple user queries from the Streamlit interface simultaneously on edge hardware without memory bottlenecking, making it highly viable for real-time educational deployment.

6.5 Unexpected Findings

The model occasionally demonstrates an understanding of mathematical principles (like commutativity) even when it fails the raw arithmetic calculation.

7 LIMITATIONS

The model is constrained by its context window size, limiting its application on extremely lengthy mathematical proofs. Furthermore, like many language models, precise large-number arithmetic remains a challenge without external tool integration.

8 CONCLUSION AND FUTURE WORK

MathSLM presents a streamlined, computationally efficient approach to mathematical reasoning using a decoder-only Transformer. Future work will focus on expanding the dataset corpus, integrating external calculator tools during the generation phase, and further refining the edge AI concurrency models to support wider deployment in interactive learning environments.

DECLARATIONS

Funding: This project received no external funding.

Conflict of Interest: The authors declare no conflicts of interest.

Ethics Statement: This research utilizes publicly available datasets.

Data Availability: Datasets (GSM8K, MATH) are accessible via Hugging Face. Code is maintained in the project repository.

Author Contributions: R.P., A.S., and N.S. contributed equally to the architecture, implementation, and documentation.

ACKNOWLEDGEMENTS

The authors thank Dr. K Swanthana for guidance and supervision throughout the development of this project.

References

- [1] Vaswani, A. et al. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

- [2] Cobbe, K. et al. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- [3] Hendrycks, D. et al. (2021). Measuring mathematical problem solving with the MATH dataset. *NeurIPS*.
- [4] Wei, J. et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.
- [5] Bai, J. et al. (2023). Qwen technical report. *arXiv preprint arXiv:2309.16609*.